For the OpenChargeMap app registration (they want to know what you're doing with their data):

The problem it solves

The Netherlands has roughly 15,000 public EV charging points. OpenChargeMap publishes data about all of them: where they are, who operates them, what power they deliver, and whether they're currently reported as working.

But that API only tells you **what is true right now**. If you ask it today, you get today's picture. Ask tomorrow, you get tomorrow's picture. Yesterday is gone — nobody stored it.

So there are questions nobody can answer from the source alone:

- Has this charger in Utrecht been broken for two days, or two months?
- Which operator has the worst reliability record?
- Did forty chargers quietly disappear from Amsterdam last month?
- Is the network growing or shrinking in my province?

Your project answers those questions by doing one simple thing the source doesn't do: **it remembers. Create API for this.**

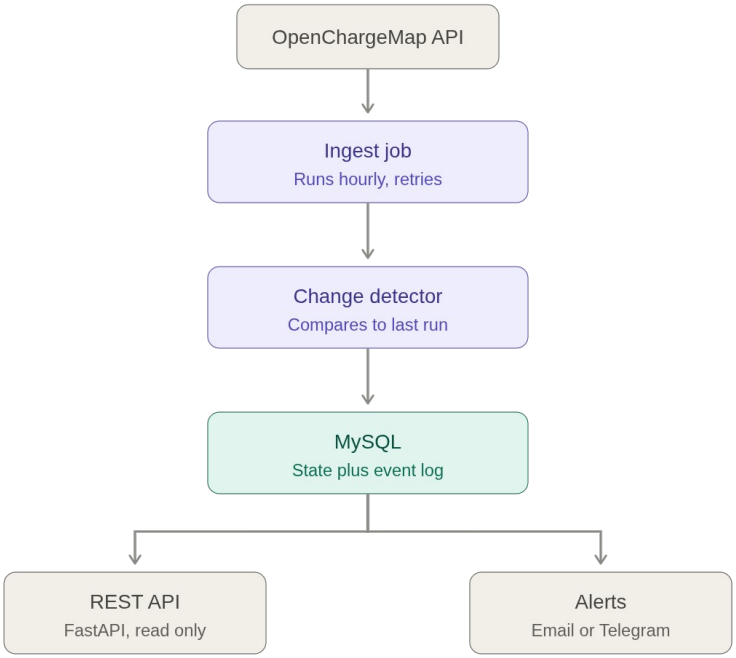
Api : "https://api.openchargemap.io/v3/poi"

The core idea

A snapshot tells you the state. A sequence of snapshots tells you the story.

Every hour, your system takes a picture of the whole Dutch network and compares it to the previous picture. Anything that changed gets written to a permanent log — a new station appeared, a station's status flipped from "operational" to "out of service", a station vanished from the feed entirely. Over weeks, that log becomes a history nobody else has.

That's the whole product. Everything else is engineering around it.



Each part, in detail

The ingest job. Wakes up on a schedule, pulls all Dutch stations from OpenChargeMap, page by page. The interesting engineering is in what happens when things go wrong: the API times out, returns a 503, or rate-limits you. Junior code crashes here. Your code retries with exponential backoff, gives up gracefully after N attempts, and records the failure in `ingest_runs` so you know a gap in your history exists and why.

The change detector. This is the brain, and it's the reason the order of operations I gave you matters. Before writing anything, it reads the previous state out of the database — that's the "before" picture. Then it compares, station by station:

- In the feed, not in the database → `new`
- In both, but `status_title` differs → `status_change`
- In the database, missing from this run's feed → `delisted`
- Was delisted, now back → `relisted`

Each difference becomes a row in `status_events`. This table is **append-only** — you never update or delete a row in it. That's a deliberate architectural choice, and it's the one worth explaining in your README: a mutable table can only ever tell you the present, while an immutable event log lets you reconstruct any past moment. It's the same reason banks store transactions instead of just a balance.

MySQL. Two tables doing two different jobs. `stations` is fast to query — "show me all broken chargers in Rotterdam" hits one indexed table. `status_events` is the history — "show me everything that happened to station 12345" walks the log. Splitting them means neither query is slow. `ingest_runs` is the third table and the one beginners skip; it's your operational record, and it's what turns a script into a *service*.

The REST API. FastAPI, read-only. Something like: list stations with filters, get one station with its full event history, list recent changes, show ingest health. Read-only is intentional — clients consume the data, only the pipeline writes it. FastAPI gives you Swagger docs for free, which means your README can link to a live, clickable API. That's a strong thing for a reviewer to land on.

Alerts. The last mile that makes it a product rather than a database. When change detection produces events matching a rule ("any station in my postcode went offline"), send a message. Telegram is the easiest to implement and the most fun to demo.

And it has a property most portfolio projects lack: **it gets more valuable while you sleep.** After a month of running, you have a dataset nobody else has. You can write a short post — "what I learned running a monitor on the Dutch charging network for 30 days" — with a real chart.

==========